Szybkie liczenie wybranych wpisów cienkiego iloczynu macierzy

Rozdział 2 pracy J. Almana i V. Vassilevskiej Williams (2026)

[Imię Nazwisko]

11 listopada 2026

J. Alman, V. Vassilevska Williams, *Truly Subquadratic 3SUM and Truly Subcubic APSP via Triangles in Sparse Lopsided Graphs*, arXiv:2610.06783, 2026.

Plan

1. Twierdzenie 5: problem i wynik
2. 2.1 Rekurencyjny algorytm Strassena
3. 2.2 Tożsamość Schönhagego
4. 2.3 Mnożenie prostokątne tożsamością Schönhagego (Coppersmith)
5. 2.4 Liczymy tylko chciane wpisy
6. 2.4.4 Dowód Twierdzenia 5
7. Podsumowanie

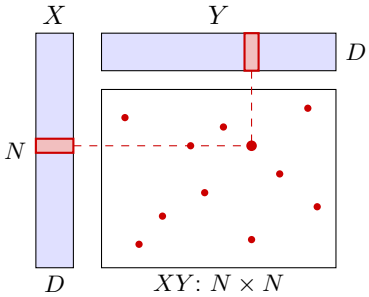
Plan

1. Twierdzenie 5: problem i wynik
2. 2.1 Rekurencyjny algorytm Strassena
3. 2.2 Tożsamość Schönhagego
4. 2.3 Mnożenie prostokątne tożsamością Schönhagego (Coppersmith)
5. 2.4 Liczymy tylko chciane wpisy
6. 2.4.4 Dowód Twierdzenia 5
7. Podsumowanie

Model obliczeń i problem

- **Model:** word RAM ze słowami długości $O(\log N)$ bitów. Liczymy *operacje* na liczbach całkowitych o $O(\log N)$ bitach.
- **Dane:** $X \in \mathbb{Z}^{N \times D}$, $Y \in \mathbb{Z}^{D \times N}$, gdzie $D \ll N$. Iloczyn XY nazywamy *cienkim*, bo macierze wejściowe są bardzo prostokątne.
- Zbiór W pozycji (I, J) macierzy $N \times N$, gdzie $I, J \in \{1, \ldots, N\}$. Pozycje z W i wpisy XY na nich nazywamy **chcianymi**.
- **Cel:** policzyć tylko

$$(XY)[I, J] = \sum_{k=1}^{D} X[I, k]\, Y[k, J] \quad \text{dla } (I, J) \in W.$$



chciany wpis = iloczyn skalarny wiersza X i kolumny Y (D mnożeń)

Główne twierdzenie rozdziału

Twierdzenie 5

Niech $D \geq 4$ będzie potęgą czwórki i niech $N \geq D^{18}$. Dla danych macierzy $X \in \mathbb{Z}^{N \times D}$ i $Y \in \mathbb{Z}^{D \times N}$, których wpisy są liczbami całkowitymi o wartości bezwzględnej co najwyżej $N^{O(1)}$, oraz zbioru W co najwyżej $N^2/\sqrt{D}$ pozycji macierzy $N \times N$, wpisy $(XY)[I, J]$ dla $(I, J) \in W$ można obliczyć deterministycznie w czasie

$$O\left(\frac{N^2 \log^2 D}{D^{1/18}}\right).$$

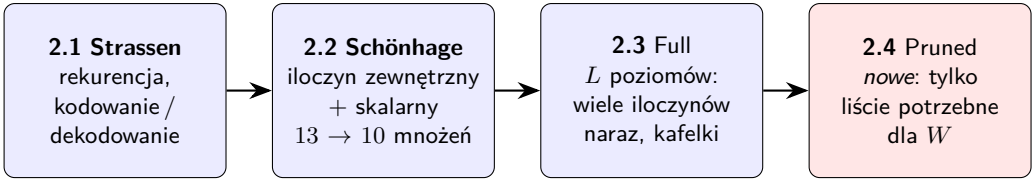
- X, Y i XY mogą być **dowolnymi gęstymi** macierzami.
- O zbiorze W zakładamy tylko tyle, że $|W| \leq N^2/\sqrt{D}$. Nie zakładamy żadnej struktury.
- Stałe (np. wykładnik 18) dobrano tak, żeby prezentacja była prosta.

Z czym się porównujemy?

Metoda	**Liczba operacji**	**Uzasadnienie**
Policzyć (wypisać) całe XY	$\geq N^2$	XY ma N^2 wpisów
Każdy chciany wpis osobno	$\lvert W\rvert \cdot D \leq \frac{N^2}{\sqrt{D}} \cdot D = N^2\sqrt{D}$	D mnożeń na wpis
Twierdzenie 5	$O\big(N^2 \log^2 D \cdot D^{-1/18}\big)$	ten rozdział

- Wynik jest **wielomianowo** (w D) mniejszy niż *wypisanie* XY i niż $N^2/\sqrt{D}$ osobnych iloczynów skalarnych.
- Przykład: przy największym dozwolonym $D = N^{1/18}$ mamy $D^{1/18} = N^{1/324}$, czyli czas $O(N^{2-1/324} \log^2 N)$.

Mapa rozdziału



- Mnożenia odbywają się w **liściach** drzewa rekursji. Dodawania na drodze w dół (*kodowanie*) i w górę (*dekodowanie*) są stosunkowo tanie.
- **Nowość 1:** kodowanie zależy *tylko od* X albo *tylko od* Y, więc liczymy je raz i współdzielimy między wieloma wywołaniami.
- **Nowość 2:** przycinamy drzewo i odwiedzamy tylko liście potrzebne chcianym wpisom. Każdy wpis ma *prywatny* liść, a pozostałe jego liście są współdzielone. Wychodzi $\approx N^2/D^{1/18}$ liści po $O(\log^2 D)$ operacji każdy.

Plan

1. Twierdzenie 5: problem i wynik
2. 2.1 Rekurencyjny algorytm Strassena
3. 2.2 Tożsamość Schönhagego
4. 2.3 Mnożenie prostokątne tożsamością Schönhagego (Coppersmith)
5. 2.4 Liczymy tylko chciane wpisy
6. 2.4.4 Dowód Twierdzenia 5
7. Podsumowanie

Tożsamość Strassena: macierze 2×2 przez 7 mnożeń

Niech $A = (a_{ij})$ i $B = (b_{ij})$ będą macierzami 2×2. Tworzymy siedem iloczynów:

$$\begin{aligned} S_1 &= (a_{11} + a_{22})(b_{11} + b_{22}), & S_5 &= (a_{11} + a_{12})\, b_{22}, \\ S_2 &= (a_{21} + a_{22})\, b_{11}, & S_6 &= (a_{21} - a_{11})(b_{11} + b_{12}), \\ S_3 &= a_{11}(b_{12} - b_{22}), & S_7 &= (a_{12} - a_{22})(b_{21} + b_{22}), \\ S_4 &= a_{22}(b_{21} - b_{11}), \end{aligned} \tag{1}$$

i wtedy

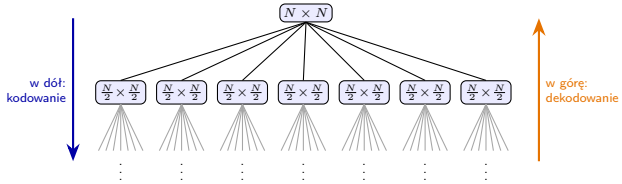
$$AB = \begin{pmatrix} S_1 + S_4 - S_5 + S_7 & S_3 + S_5 \\ S_2 + S_4 & S_1 - S_2 + S_3 + S_6 \end{pmatrix}.$$

- Każde S_r to (kombinacja liniowa wpisów A) $\times$ (kombinacja liniowa wpisów B).
- Każdy wpis AB to kombinacja liniowa $S_1, \ldots, S_7$.
- **7 mnożeń zamiast 8.**

Rekurencja na blokach

- A, B rozmiaru $N \times N$, gdzie $N = 2^L$.
- Traktujemy je jak macierze 2×2 *bloków* rozmiaru $\frac{N}{2} \times \frac{N}{2}$.
- Tożsamość nie korzysta z przemienności mnożenia, więc działa dla bloków.
- Tworzymy 7 par kombinacji bloków, każdą parę mnożymy *rekurencyjnie*, a z 7 wyników składamy 4 bloki AB.
- Drzewo ma L poziomów i 7^L liści, po jednym na każdy wybór jednego z 7 iloczynów na każdym poziomie.
- Mnożenia są **w liściach**:

$$7^L = N^{\log_2 7} < N^{2.81}.$$



7^L liści: iloczyny dwóch liczb

Równoważnie: $T(N) = 7\,T(N/2) + O(N^2)$.

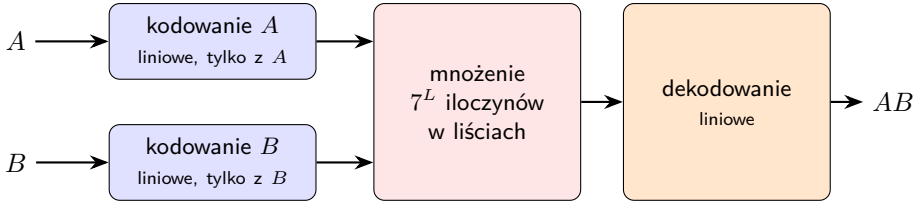
Koszt dodawań i odejmowań

- Na głębokości k drzewa jest 7^k wierzchołków wewnętrznych.
- W każdym z nich, w dół i w górę, liczymy kombinacje liniowe macierzy $2^{L-k} \times 2^{L-k}$, czyli $O(4^{L-k})$ operacji.

$$\sum_{k=0}^{L-1} 7^k 4^{L-k} \;=\; 4^L \sum_{k=0}^{L-1} \left(\tfrac{7}{4}\right)^k \;=\; 4^L \cdot \frac{(7/4)^L - 1}{7/4 - 1} \;<\; \tfrac{4}{3} \cdot 7^L \;=\; O(7^L).$$

Szereg geometryczny jest zdominowany przez ostatni wyraz, więc dodawania zwiększają czas tylko o stały czynnik. Uwaga: w nowym algorytmie (2.4) ten argument *przestanie działać*. Dodawania trzeba będzie liczyć staranniej.

Wnioski: trzy fazy



- Kodowanie i dekodowanie są **liniowe**.
- Kodowanie A **nie zależy od** B (i odwrotnie). Wykorzystamy to w 2.4.1.
- Tutaj kodowanie i dekodowanie są tańsze niż mnożenia, więc można je zignorować. W pełnym algorytmie (2.4) już nie!

Kluczowa idea. Mała tożsamość stosowana rekurencyjnie: mnożenia w liściach, a ich liczba decyduje o koszcie.

Notacja: wielomiany trójliniowe

- Wprowadzamy *zmienne wyjściowe* $c_{11}, c_{12}, c_{21}, c_{22}$, po jednej na każdy wpis AB.
- Każde S_r mnożymy przez sumę ze znakami tych c_{ik}, w których wzorach S_r występuje. Na przykład S_1 jest we wzorach na $(AB)_{11}$ i $(AB)_{22}$, więc dostaje $c_{11} + c_{22}$. S_5 jest z minusem w $(AB)_{11}$ i z plusem w $(AB)_{12}$, więc dostaje $c_{12} - c_{11}$.

$$S_1(c_{11}+c_{22})+S_2(c_{21}-c_{22})+S_3(c_{12}+c_{22})+S_4(c_{11}+c_{21})+S_5(c_{12}-c_{11})+S_6c_{22}+S_7c_{11} = \sum_{i,j,k=1}^{2} a_{ij}b_{jk}c_{ik}.$$

- Współczynnik przy c_{ik} **po prawej** to szukane $(AB)_{ik} = a_{i1}b_{1k} + a_{i2}b_{2k}$.
- **Po lewej** to kombinacja, która go liczy. Dla c_{11} jest to $S_1 + S_4 - S_5 + S_7$.
- Każdy składnik ma postać $\varphi(a) \cdot \psi(b) \cdot \chi(c)$, czyli (forma liniowa w a) $\times$ (forma w b) $\times$ (forma w c). Jedna tożsamość zapisuje *kodowanie*, *mnożenia* i *dekodowanie* naraz.
- Wielomian jest *trójliniowy*: liniowy osobno w a, w b i w c. W literaturze nazywa się go tensorem rzędu 3.

Plan

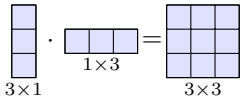
1. Twierdzenie 5: problem i wynik
2. 2.1 Rekurencyjny algorytm Strassena
3. 2.2 Tożsamość Schönhagego
4. 2.3 Mnożenie prostokątne tożsamością Schönhagego (Coppersmith)
5. 2.4 Liczymy tylko chciane wpisy
6. 2.4.4 Dowód Twierdzenia 5
7. Podsumowanie

Dwa iloczyny na rozłącznych zmiennych

Iloczyn zewnętrzny wektorów (x_1, x_2, x_3) i (y_1, y_2, y_3):

dziewięć liczb $x_i y_j, \quad 1 \le i, j \le 3.$

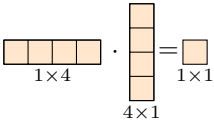
Jest to iloczyn macierzy 3×1 przez 1×3.



Iloczyn skalarny wektorów $(p_{11}, p_{12}, p_{21}, p_{22})$ i $(q_{11}, q_{12}, q_{21}, q_{22})$:

$$\sum_{i,j=1}^{2} p_{ij} q_{ij}.$$

Jest to iloczyn macierzy 1×4 przez 4×1.



Indeksy $\{1,2\}^2$ zamiast $\{1,2,3,4\}$ ułatwią zapis $\hat{p}$, $\hat{q}$.

- Osobno: $9 + 4 = \mathbf{13}$ mnożeń.
- **Schönhage [Sch81]:** wystarczy **10** mnożeń, jeśli dopuścimy pewne wyrazy błędu (okażą się nieszkodliwe).

Zapis trójliniowy: wielomian G

- **7 zmiennych lewych:** x_1, x_2, x_3, $p_{11}, p_{12}, p_{21}, p_{22}$.
- **7 zmiennych prawych:** y_1, y_2, y_3, $q_{11}, q_{12}, q_{21}, q_{22}$.
- **10 zmiennych wyjściowych:** z_{ij} $(1 \leq i, j \leq 3)$ i z_0, po jednej na każdą z dziesięciu liczb do policzenia.

$$G := \sum_{i,j=1}^{3} x_i y_j \, z_{ij} \; + \; \Big(\sum_{i,j=1}^{2} p_{ij} q_{ij} \Big) z_0.$$

- Współczynnik przy z_{ij} to $x_i y_j$ (iloczyn zewnętrzny).
- Współczynnik przy z_0 to $\sum p_{ij} q_{ij}$ (iloczyn skalarny).
- Chcemy zapisać G (z dokładnością do błędu) jako sumę **dziesięciu** wyrazów postaci (forma liniowa w zmiennych lewych)$\times$(forma w prawych)$\times$(forma w wyjściowych).

Macierze $\hat{p}$ i $\hat{q}$

Rozszerzamy macierze 2×2 zmiennych p i q do 3×3 tak, żeby **każda kolumna** $\hat{p}$ i **każdy wiersz** $\hat{q}$ sumowały się do zera:

$$\hat{p} = \begin{pmatrix} p_{11} & p_{12} & 0 \\ p_{21} & p_{22} & 0 \\ -p_{11}-p_{21} & -p_{12}-p_{22} & 0 \end{pmatrix}, \qquad \hat{q} = \begin{pmatrix} q_{11} & q_{12} & -q_{11}-q_{12} \\ q_{21} & q_{22} & -q_{21}-q_{22} \\ 0 & 0 & 0 \end{pmatrix}.$$

- Elementy $\hat{p}_{ij}$, $\hat{q}_{ij}$ to *formy liniowe*, a nie nowe zmienne.
- $\sum_{i,j=1}^{3} \hat{p}_{ij}\hat{q}_{ij} = \sum_{i,j=1}^{2} p_{ij}q_{ij}$, bo trzecia kolumna $\hat{p}$ i trzeci wiersz $\hat{q}$ są zerowe, a dla $i, j \leq 2$ mamy $\hat{p}_{ij} = p_{ij}$, $\hat{q}_{ij} = q_{ij}$.

Dziesięć składników i ich formy liniowe

Składniki to P_{ij} (dla $1 \le i, j \le 3$, odpowiadające wpisom $\hat{p}$ i $\hat{q}$) oraz P_0. Każdy **składnik** λ jest iloczynem trzech form liniowych: φ_λ w zmiennych lewych, ψ_λ w prawych, χ_λ w wyjściowych.

składnik λ	φ_λ (lewe)	ψ_λ (prawe)	χ_λ (wyjściowe)
P_{ij}, $1 \le i, j \le 3$	$x_i + \hat{p}_{ij}$	$y_j + \hat{q}_{ij}$	$z_{ij} + z_0$
P_0	$-(x_1 + x_2 + x_3)$	$y_1 + y_2 + y_3$	z_0

- 10 składników to **10 mnożeń** $\varphi_\lambda \cdot \psi_\lambda$.
- Każde φ_λ i każde ψ_λ ma **co najwyżej 3** jednomiany o współczynnikach ± 1.

Lemat 6: tożsamość Schönhagego

Lemat 6

Sumując po dziesięciu składnikach,

$$\sum_{\lambda} \varphi_\lambda \psi_\lambda \chi_\lambda = G + E, \qquad \textit{gdzie} \qquad E := \sum_{i,j=1}^{3} \left(x_i \hat{q}_{ij} + \hat{p}_{ij} y_j + \hat{p}_{ij} \hat{q}_{ij} \right) z_{ij}.$$

- Lewa strona: 10 wyrazów, czyli **10 mnożeń** $\varphi_\lambda \cdot \psi_\lambda$.
- Współczynnik przy z_0 po lewej to **dokładnie** iloczyn skalarny $\sum p_{ij} q_{ij}$.
- Współczynnik przy z_{ij} to $x_i y_j$ **plus błąd** $x_i \hat{q}_{ij} + \hat{p}_{ij} y_j + \hat{p}_{ij} \hat{q}_{ij}$.
- Lemat 6 to kluczowa tożsamość: liczy chciane G plus wielomian błędu E.

Dowód Lematu 6

Dowód.

Porównujemy współczynniki przy każdej zmiennej wyjściowej po lewej stronie.

- **Przy z_{ij}:** z_{ij} występuje tylko w $\chi_{P_{ij}}$, więc współczynnik wynosi

$$\varphi_{P_{ij}}\psi_{P_{ij}} = (x_i + \hat{p}_{ij})(y_j + \hat{q}_{ij}) = x_i y_j + \big(x_i\hat{q}_{ij} + \hat{p}_{ij}y_j + \hat{p}_{ij}\hat{q}_{ij}\big).$$

- **Przy z_0:** z_0 występuje we wszystkich χ_λ, więc współczynnik wynosi

$$\sum_{i,j=1}^{3}(x_i+\hat{p}_{ij})(y_j+\hat{q}_{ij}) - (x_1+x_2+x_3)(y_1+y_2+y_3) = \underbrace{\sum_i x_i \sum_j \hat{q}_{ij}}_{=0\ \text{(wiersze } \hat{q})} + \underbrace{\sum_j y_j \sum_i \hat{p}_{ij}}_{=0\ \text{(kolumny } \hat{p})} + \sum_{i,j=1}^{3}\hat{p}_{ij}\hat{q}_{ij}.$$

Iloczyny $x_i y_j$ się skracają, a $\sum_{i,j=1}^{3}\hat{p}_{ij}\hat{q}_{ij} = \sum_{i,j=1}^{2} p_{ij}q_{ij}$. □

Obserwacja 1: zmienne wewnętrzne i zewnętrzne

Definicja

Zmienne iloczynu skalarnego, czyli p_{ij}, q_{ij}, z_0, nazywamy **wewnętrznymi**. Zmienne iloczynu zewnętrznego, czyli x_i, y_j, z_{ij}, nazywamy **zewnętrznymi**.

Obserwacja

- Każdy jednomian G jest *czysto* wewnętrzny ($p_{ij}q_{ij}z_0$) albo *czysto* zewnętrzny ($x_i y_j z_{ij}$).
- Każdy jednomian E ma zewnętrzną zmienną wyjściową z_{ij} razem z wewnętrzną zmienną lewą lub prawą: $x_i\hat{q}_{ij}z_{ij}$, $\hat{p}_{ij}y_j z_{ij}$, $\hat{p}_{ij}\hat{q}_{ij}z_{ij}$.

To pozwoli pokazać, że E jest nieszkodliwe (Lemat 9).

Obserwacja 2: kto wnosi wkład do którego wyjścia

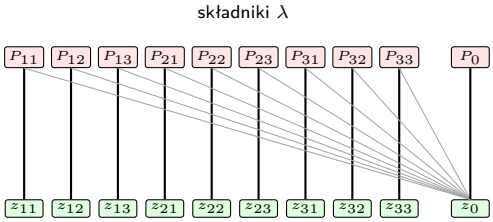
Definicja

Składnik λ **wnosi wkład** do zmiennej wyjściowej z, jeśli z występuje w χ_λ.

- **Każdy** składnik wnosi wkład do z_0.
- Do z_{ij} wnosi wkład **tylko** P_{ij}.
- Formy χ_λ są sumami zmiennych bez współczynników, więc współczynnik przy z po lewej stronie to

$$\sum_{\lambda \text{ wnosi wkład do } z} \varphi_\lambda \psi_\lambda.$$

- To, że do z_{ij} wnosi wkład tylko jeden składnik, pozwoli pominąć wiele mnożeń, gdy chcemy tylko niektórych wyjść.



Plan

1. Twierdzenie 5: problem i wynik
2. 2.1 Rekurencyjny algorytm Strassena
3. 2.2 Tożsamość Schönhagego
4. 2.3 Mnożenie prostokątne tożsamością Schönhagego (Coppersmith)
5. 2.4 Liczymy tylko chciane wpisy
6. 2.4.4 Dowód Twierdzenia 5
7. Podsumowanie

Idea: L poziomów rekursji daje 2^L iloczynów

- Tożsamość Schönhagego liczy dwa iloczyny na rozłącznych zmiennych: zewnętrzny (kształt 3×1 razy 1×3) i skalarny (1×4 razy 4×1).
- Stosowana rekurencyjnie na L poziomach daje 2^L iloczynów macierzy, po jednym na każdy wybór zewnętrzny/skalarny na każdym poziomie.
- Poziom zewnętrzny mnoży liczbę wierszy i kolumn wyniku przez 3. Poziom skalarny mnoży wymiar wewnętrzny (długość sum) przez 4.
- Wybór skalarnego na m poziomach i zewnętrznego na pozostałych $L - m$ daje iloczyn macierzy

$$3^{L-m} \times 4^m \quad \text{przez} \quad 4^m \times 3^{L-m}.$$

Interesuje nas m **dużo mniejsze** od L, czyli cienkie iloczyny.

Przykład ($L = 2$)

zewn., zewn.	zewn., skal.	skal., zewn.	skal., skal.
9×1 razy 1×9	3×4 razy 4×3	3×4 razy 4×3	1×16 razy 16×1
$(m = 0)$	$(m = 1)$	$(m = 1)$	$(m = 2)$

Notacja: napisy i tablice

Definicja

- **Lewy napis** długości L to ciąg $u = u_1 u_2 \cdots u_L$ złożony z L zmiennych lewych. u_ℓ to jego zmienna na **poziomie** ℓ.
- **Tablica** a na lewych napisach długości L przypisuje każdemu takiemu u liczbę całkowitą $a[u]$. Ma ona 7^L wpisów.
- Tak samo definiujemy napisy i tablice **prawe** (7^L wpisów) i **wyjściowe** (10^L wpisów).

Definicja

Dla składnika λ i zmiennej lewej s niech $\varphi_\lambda(s) \in \{0, \pm 1\}$ oznacza współczynnik przy s w φ_λ. Analogicznie $\psi_\lambda(t)$ dla zmiennej prawej t.

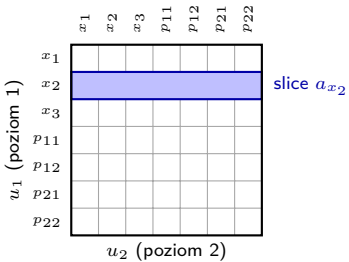
Slice: odpowiednik bloków macierzy

Definicja

Dla tablicy a na napisach długości $L \geq 1$ i zmiennej lewej s **slice** a_s to tablica na napisach długości $L - 1$:

$$a_s[u'] := a[su'].$$

- a składa się z **siedmiu** slice'ów, tak jak macierz blokowa składa się z bloków.
- Analogicznie slice'y b_t tablicy prawej i c_z tablicy wyjściowej (**dziesięć** slice'ów).
- Slice wybieramy według zmiennej na **poziomie 1**.



tablica a dla $L = 2$: $7^2 = 49$ wpisów

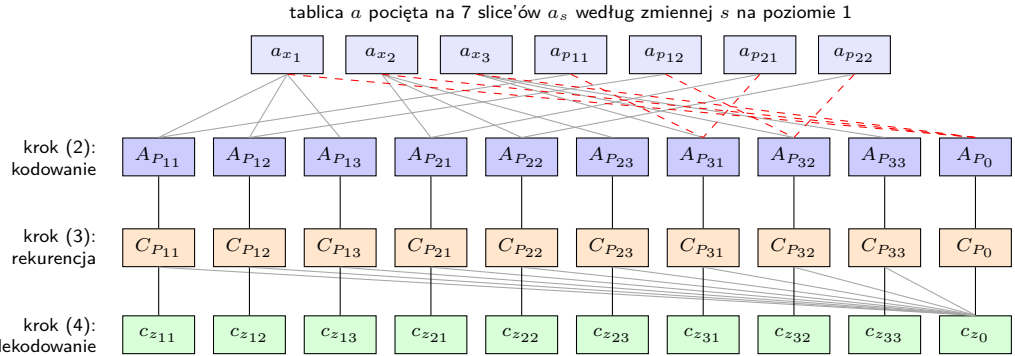
Algorytm Full(a, b)

Full(a, b) dla tablic a, b na lewych i prawych napisach długości L; zwraca tablicę na wyjściowych napisach długości L

1. Jeśli $L = 0$, zwróć liczbę ab.
2. **(kodowanie)** Dla każdego składnika λ utwórz
$$A_\lambda := \sum_s \varphi_\lambda(s)\, a_s, \qquad B_\lambda := \sum_t \psi_\lambda(t)\, b_t.$$
3. **(rekurencja)** Dla każdego λ policz rekurencyjnie $C_\lambda := \text{Full}(A_\lambda, B_\lambda)$.
4. **(dekodowanie)** Zwróć tablicę c o slice'ach
$$c_{z_{ij}} := C_{P_{ij}}, \qquad c_{z_0} := \sum_\lambda C_\lambda.$$

- Dla $L = 1$ jest to dokładnie jedno zastosowanie tożsamości Schönhagego: krok (2) liczy formy φ_λ, ψ_λ, krok (3) mnoży, a krok (4) sumuje według χ_λ.
- Krok (4) to χ_λ: do z_{ij} wnosi wkład tylko P_{ij}, a do z_0 wszystkie.

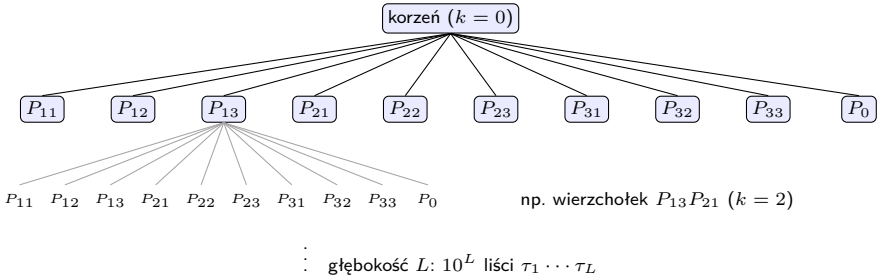
Jeden poziom rekursji



- Każdy A_λ łączy co najwyżej 3 slice'y (linia ciągła: $+$, przerywana: $-$), np. $A_{P_{31}} = a_{x_3} - a_{p_{11}} - a_{p_{21}}$, $A_{P_0} = -a_{x_1} - a_{x_2} - a_{x_3}$. To samo robimy z b i ψ_λ.
- Dziesięć par mnożymy wywołaniami rekurencyjnymi $C_\lambda = \mathsf{Full}(A_\lambda, B_\lambda)$.
- $c_{z_{ij}} = C_{P_{ij}}$, a c_{z_0} to suma wszystkich dziesięciu, bo każdy składnik wnosi wkład do z_0.

Drzewo rekursji Full

- Wywołanie osiągnięte przez wybór składników $\tau_1, \ldots, \tau_k$ na poziomach $1, \ldots, k$ nazywamy **wierzchołkiem** $\tau_1 \cdots \tau_k$ (napis k składników). Leży on na głębokości k, a jego tablice wejściowe są na napisach długości $L - k$.
- Korzeń ma $k = 0$. **Liście** $\tau = \tau_1 \cdots \tau_L$ (głębokość L) to 10^L mnożeń z kroku (1).



Koszt Full

Koszt. Na głębokości k jest 10^k wierzchołków. Każdy ma dwie tablice wejściowe po 7^{L-k} wpisów i tablicę wyjściową z 10^{L-k} wpisami. Krok (2) kosztuje $O(7^{L-k})$ (dziesięć kombinacji co najwyżej trzech slice'ów po 7^{L-k-1} wpisów), a krok (4) $O(10^{L-k})$:

$$\sum_{k=0}^{L} 10^k \cdot O\big(7^{L-k} + 10^{L-k}\big) = O(L \cdot 10^L).$$

- Kodowanie: $\displaystyle\sum_{k=0}^{L} 10^k 7^{L-k} = 10^L \sum_{k=0}^{L} \left(\tfrac{7}{10}\right)^{L-k} < \tfrac{10}{3} \cdot 10^L = O(10^L)$.
- Dekodowanie: $\displaystyle\sum_{k=0}^{L} 10^k 10^{L-k} = (L+1)\,10^L$. To ono dominuje.

Kluczowa idea. Full kosztuje niewiele więcej niż 10^L mnożeń w liściach: kodowanie $O(10^L)$, dekodowanie $O(L \cdot 10^L)$.

Co dokładnie liczy Full? Rozwinięcie rekurencji

Definicja

Dla liścia τ:

$$\Phi_\tau(a) := \sum_u a[u] \prod_{\ell=1}^{L} \varphi_{\tau_\ell}(u_\ell), \qquad \Psi_\tau(b) := \sum_v b[v] \prod_{\ell=1}^{L} \psi_{\tau_\ell}(v_\ell),$$

sumy po wszystkich lewych napisach u i prawych napisach v długości L.

- $\Phi_\tau(a)$ to liczba, którą kodowanie dostarcza do liścia τ: na poziomie ℓ stosujemy formę φ_{τ_ℓ}.
- Przykład $L = 1$: $\Phi_\lambda(a) = \sum_s \varphi_\lambda(s)\, a[s]$, czyli forma φ_λ wyliczona na siedmiu liczbach $a[s]$.
- Przykład $L = 2$: $\Phi_{P_0 P_{11}}(a) = \sum_{s,s'} \varphi_{P_0}(s)\varphi_{P_{11}}(s')\, a[ss'] = -\sum_{i=1}^{3} \big(a[x_i x_1] + a[x_i p_{11}]\big)$.

Wkład wierzchołka do napisu i tablica Mult

Definicja

Wierzchołek $\tau_1 \cdots \tau_k$ **wnosi wkład** do napisu wyjściowego w, jeśli τ_ℓ wnosi wkład do w_ℓ dla każdego poziomu $\ell \leq k$. Zatem liść τ wnosi wkład do w wtedy i tylko wtedy, gdy $\tau_\ell = P_{ij}$ na każdym poziomie ℓ, na którym $w_\ell = z_{ij}$. Tam, gdzie $w_\ell = z_0$, τ_ℓ jest dowolne.

$$\mathsf{Mult}(a,b)[w] := \sum_{\tau \text{ wnosi wkład do } w} \Phi_\tau(a)\,\Psi_\tau(b). \tag{2}$$

Przykład ($L = 2$)

Do $w = z_{13}z_0$ wnosi wkład 10 liści $P_{13}\lambda$ (dowolne λ). Do $w = z_{13}z_{21}$ tylko liść $P_{13}P_{21}$. Do $w = z_0z_0$ wszystkie 100 liści.

Lemat 7

Lemat 7

W liściu τ algorytm Full(a, b) *mnoży* $\Phi_\tau(a)$ *przez* $\Psi_\tau(b)$ *i zwraca* Mult(a, b).

Dowód.

Indukcja po L. $L = 0$**:** jedynym liściem jest pusty napis τ. Iloczyny po poziomach są puste, więc $\Phi_\tau(a) = a$, $\Psi_\tau(b) = b$, a krok (1) zwraca $ab =$ Mult(a, b).
$L \geq 1$**, kodowanie:** dla liścia $\lambda\tau'$ (składnik λ, potem napis τ' długości $L - 1$), ponieważ $a_s[u'] = a[su']$:

$$\Phi_{\lambda\tau'}(a) = \sum_s \varphi_\lambda(s) \sum_{u'} a[su'] \prod_{\ell=1}^{L-1} \varphi_{\tau'_\ell}(u'_\ell) = \sum_s \varphi_\lambda(s)\, \Phi_{\tau'}(a_s) = \Phi_{\tau'}\Big(\sum_s \varphi_\lambda(s) a_s\Big) = \Phi_{\tau'}(A_\lambda).$$

Podobnie $\Psi_{\lambda\tau'}(b) = \Psi_{\tau'}(B_\lambda)$. Z założenia indukcyjnego liść $\lambda\tau'$ mnoży $\Phi_{\lambda\tau'}(a)$ przez $\Psi_{\lambda\tau'}(b)$ oraz $C_\lambda[w'] = \sum_{\tau'} \Phi_{\lambda\tau'}(a)\Psi_{\lambda\tau'}(b)$, suma po τ' wnoszących wkład do w'.
Dekodowanie: do zw' wnoszą wkład dokładnie liście $\lambda\tau'$, gdzie λ wnosi wkład do z, a τ' do w'. Zatem $c[zw'] = \sum_{\lambda \text{ wnosi wkład do } z} C_\lambda[w']$ zgadza się z (2). □

Współczynniki γ i Lemat 8

Dla zmiennej lewej s, prawej t i wyjściowej z niech

$$\gamma(s,t,z) := \sum_{\lambda \text{ wnosi wkład do } z} \varphi_\lambda(s)\,\psi_\lambda(t). \tag{3}$$

To współczynnik przy jednomianie stz po lewej stronie tożsamości. Z Lematu 6 jest to też współczynnik przy stz w $G + E$, więc $\gamma(s,t,z) \in \{0, \pm 1\}$.

Lemat 8

Dla wszystkich tablic a, b i każdego napisu wyjściowego w:

$$\mathsf{Mult}(a,b)[w] = \sum_{u,v} \gamma(u_1,v_1,w_1)\,\gamma(u_2,v_2,w_2)\cdots\gamma(u_L,v_L,w_L)\; a[u]\,b[v],$$

gdzie suma przebiega po wszystkich lewych napisach u i prawych napisach v.

Dowód Lematu 8

Dowód.

Z definicji Φ_τ, Ψ_τ: $\Phi_\tau(a)\Psi_\tau(b) = \sum_{u,v} a[u]b[v] \prod_{\ell=1}^{L} \varphi_{\tau_\ell}(u_\ell)\psi_{\tau_\ell}(v_\ell)$. Zatem współczynnik przy $a[u]b[v]$ w $\mathrm{Mult}(a,b)[w]$ to

$$\sum_{\tau \text{ wnosi wkład do } w} \prod_{\ell=1}^{L} \varphi_{\tau_\ell}(u_\ell)\psi_{\tau_\ell}(v_\ell).$$

Liście wnoszące wkład do w to te, których składnik na poziomie ℓ wnosi wkład do w_ℓ, **niezależnie dla każdego** ℓ. Suma iloczynów jest więc iloczynem sum:

$$\prod_{\ell=1}^{L} \sum_{\tau_\ell \text{ wnosi wkład do } w_\ell} \varphi_{\tau_\ell}(u_\ell)\psi_{\tau_\ell}(v_\ell) \overset{(3)}{=} \prod_{\ell=1}^{L} \gamma(u_\ell, v_\ell, w_\ell).$$

□

Dwa opisy Mult**:**

- Lemat 8 posłuży do odczytania iloczynów macierzy (2.3.3).
- Lemat 7 (suma po liściach) posłuży do pomijania liści, które nie wnoszą wkładu do chcianych wpisów (2.4).

Zbiór wewnętrzny, część wewnętrzna i zewnętrzna

Definicja

- **Zbiór wewnętrzny** napisu to zbiór poziomów, na których ma on zmienną wewnętrzną: p (zamiast x) dla lewego, q (zamiast y) dla prawego, z_0 (zamiast z_{ij}) dla wyjściowego.
- **Część wewnętrzna** to napis jego zmiennych na poziomach zbioru wewnętrznego (w kolejności poziomów), a **część zewnętrzna** to napis zmiennych na pozostałych poziomach.

Napis jest wyznaczony przez swój zbiór wewnętrzny, część zewnętrzną i część wewnętrzną.

$u =$

1	2	3	4	5	6
x_1	p_{11}	x_2	x_3	p_{22}	x_1

zbiór wewnętrzny $Q = \{2, 5\}$, część wewnętrzna $p_{11}p_{22}$, część zewnętrzna $x_1x_2x_3x_1$ $\quad (L = 6)$

Parametry i indeksowanie macierzy napisami

Ustalmy $m \geq 1$ oraz

$$N_0 := 3^{L-m}, \qquad D := 4^m.$$

- Z 2^L iloczynów (po jednym na zbiór $Q \subseteq \{1, \ldots, L\}$ poziomów ze skalarnym) używamy tylko $\binom{L}{m}$ tych z $|Q| = m$. Wejścia pozostałych ustawiamy na zero.
- Iloczyn dla takiego Q to macierz $N_0 \times D$ razy $D \times N_0$: $L - m$ poziomów zewnętrznych (po 3 zmienne) daje $N_0 = 3^{L-m}$ wierszy i kolumn, a m poziomów wewnętrznych (po 4 zmienne) daje $D = 4^m$ składników sumy.

Indeksowanie:

- Macierz $N_0 \times D$: wiersze to napisy z $L - m$ zewnętrznych lewych zmiennych (x), kolumny to napisy z m wewnętrznych lewych (p).
- Macierz $D \times N_0$: wiersze to napisy z m wewnętrznych prawych (q), kolumny to napisy z zewnętrznych prawych (y).
- Kolumnę $\pi = p_{i_1 j_1} \cdots p_{i_m j_m}$ łączymy z wierszem $\pi' = q_{i_1 j_1} \cdots q_{i_m j_m}$ (te same indeksy):

$$(AB)[r, c] := \sum_{\pi} A[r, \pi]\, B[\pi', c],$$

suma po wszystkich D napisach π. Wiersze r wyniku to napisy z x, a kolumny c to napisy z y.

Kodowanie macierzy w tablicy

Niech $Q \subseteq \{1, \ldots, L\}$, $|Q| = m$, i $K := \binom{L}{m}$.

- Lewe napisy u o zbiorze wewnętrznym Q indeksują wpisy macierzy X_Q wymiaru $N_0 \times D$: $X_Q[u]$ to wpis w wierszu (część zewnętrzna u) i kolumnie (część wewnętrzna u).
- Prawe napisy v o zbiorze wewnętrznym Q indeksują Y_Q wymiaru $D \times N_0$: wiersz to część wewnętrzna, kolumna to część zewnętrzna.
- Napis wyjściowy w o zbiorze wewnętrznym Q ze zmiennymi $z_{i_1 j_1}, \ldots, z_{i_{L-m} j_{L-m}}$ poza Q wskazuje wpis $(X_Q Y_Q)[w]$ w wierszu $x_{i_1} \cdots x_{i_{L-m}}$ i kolumnie $y_{j_1} \cdots y_{j_{L-m}}$.

Wejście: macierze ułożone obok siebie

Dla każdego z K zbiorów Q bierzemy *dowolne* X_Q $(N_0 \times D)$ i Y_Q $(D \times N_0)$ i ustawiamy

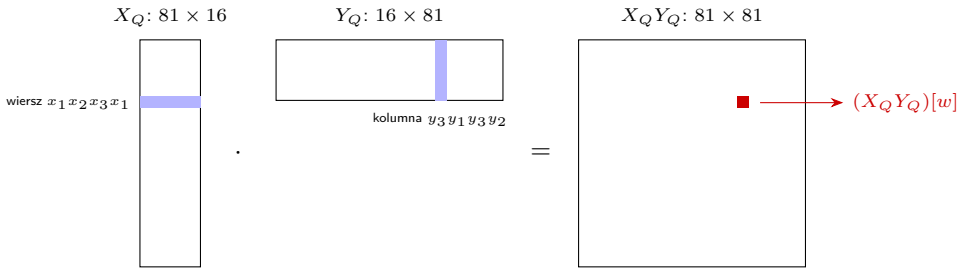
$$a[u] := X_Q[u], \qquad b[v] := Y_Q[v]$$

dla napisów o zbiorze wewnętrznym Q z $|Q| = m$, a $a[u] := 0$, $b[v] := 0$ dla wszystkich pozostałych napisów.

Od napisów do wpisów macierzy ($L = 6$, $m = 2$)

u:	x_1	p_{11}	x_2	x_3	p_{22}	x_1	$x_1x_2x_3x_1$: wiersz u w X_Q, $p_{11}p_{22}$: kolumna
v:	y_3	q_{11}	y_1	y_3	q_{22}	y_2	$q_{11}q_{22}$: wiersz v w Y_Q, $y_3y_1y_3y_2$: kolumna
w:	z_{13}	z_0	z_{21}	z_{33}	z_0	z_{12}	wiersz $x_1x_2x_3x_1$ i kolumna $y_3y_1y_3y_2$ odczytane z z_{ij}

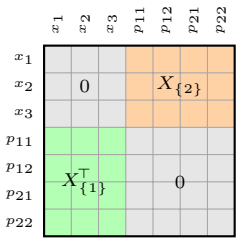
poziomy $\ell = 1, \ldots, 6$; cieniowane: $Q = \{2, 5\}$



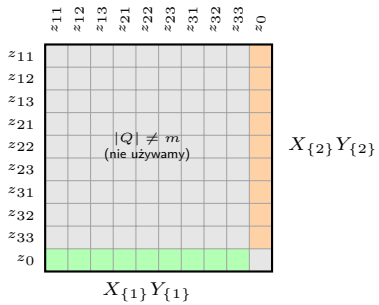
Jedno uruchomienie Full liczy wszystkie $\binom{6}{2} = 15$ iloczynów X_QY_Q, po jednym na każdy Q rozmiaru 2.

Przykład $L = 2$, $m = 1$: gdzie leżą macierze w tablicach

$N_0 = 3$, $D = 4$, $K = \binom{2}{1} = 2$: dwa iloczyny 3×4 razy 4×3, dla $Q = \{1\}$ i $Q = \{2\}$.



tablica a: wiersz u_1, kolumna u_2



tablica wyników: wiersz w_1, kolumna w_2

- $u = x_i p_{jk}$ ma zbiór wewnętrzny $\{2\}$ i wiersz x_i, kolumnę p_{jk} w $X_{\{2\}}$. Napis $u = p_{jk} x_i$ ma $\{1\}$, a x_i jest teraz na poziomie 2 (stąd transpozycja na rysunku).
- Napis $w = z_{ij} z_0$ to wpis $(X_{\{2\}} Y_{\{2\}})[x_i, y_j]$, a $w = z_0 z_{ij}$ to wpis $(X_{\{1\}} Y_{\{1\}})[x_i, y_j]$.

Lemat 9: Full naprawdę mnoży macierze

Lemat 9

Dla tablic wejściowych a, b zdefiniowanych powyżej i każdego napisu wyjściowego w, którego zbiór wewnętrzny Q ma dokładnie m elementów,

$$\mathsf{Mult}(a, b)[w] = (X_Q Y_Q)[w]. \tag{4}$$

Dowód (1/2).

Z Lematu 8: $\mathsf{Mult}(a, b)[w] = \sum_{u,v} \gamma(u_1, v_1, w_1) \cdots \gamma(u_L, v_L, w_L)\, a[u] b[v]$, gdzie $\gamma(s, t, z)$ to współczynnik przy stz w $G + E$. Sprawdzamy, które składniki mogą być niezerowe.

1. $a[u]b[v] \neq 0$ wymaga, by zbiory wewnętrzne u i v miały **dokładnie** m elementów.
2. **Poziom** $\ell \in Q$**:** $w_\ell = z_0$. Jedyne jednomiany $G + E$ z z_0 to $p_{ij} q_{ij} z_0$ (współczynnik 1), bo E zawiera tylko z_{ij}. Zatem $(u_\ell, v_\ell) = (p_{ij}, q_{ij})$ dla pewnych i, j. Czyli u i v mają zmienną wewnętrzną na każdym poziomie Q, a ich zbiory wewnętrzne (rozmiaru m) są **równe** Q.

□

Dowód Lematu 9 (2/2) i dlaczego E nie szkodzi

Dowód (2/2).

3. **Poziom** $\ell \notin Q$**:** $w_\ell = z_{ij}$, a u_ℓ, v_ℓ są zewnętrzne (zbiory wewnętrzne to dokładnie Q). Jedyny jednomian $G + E$ z z_{ij} *bez zmiennej wewnętrznej* to $x_i y_j z_{ij}$ (współczynnik 1). Zatem $(u_\ell, v_\ell) = (x_i, y_j)$.
4. Zostają składniki, w których u ma zbiór wewnętrzny Q, część zewnętrzną równą wierszowi r napisu w i jakąś część wewnętrzną π, a v ma zbiór wewnętrzny Q, część zewnętrzną równą kolumnie c napisu w i część wewnętrzną π' o tych samych indeksach co π. Jest jeden taki składnik na każdy z D napisów π, ze współczynnikiem 1:

$$\sum_{\pi} X_Q[r, \pi]\, Y_Q[\pi', c] = (X_Q Y_Q)[w].$$

□

Dlaczego E nie szkodzi: podsumowanie 2.3.3

Kluczowa idea. Każdy jednomian E ma wewnętrzną zmienną wejściową przy zewnętrznym z_{ij}. Na poziomie spoza Q dałoby to u lub v o zbiorze wewnętrznym rozmiaru $> m$, a tam $a[u]b[v] = 0$. **Błąd E ginie.**

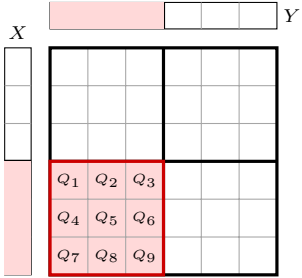
Przykład ($L = 2$, $m = 1$, $w = z_{21}z_0$, czyli $Q = \{2\}$)

Jednomian $x_2\hat{q}_{21}z_{21} = x_2q_{21}z_{21}$ z E daje $\gamma(x_2, q_{21}, z_{21}) = 1$. Ale na poziomie 2 ($w_2 = z_0$) napis v musi mieć q_{jk}, więc $v = q_{21}q_{jk}$ ma zbiór wewnętrzny $\{1, 2\}$ rozmiaru $2 \neq m$. Zatem $b[v] = 0$ i ten składnik sumy z Lematu 8 znika.

Podsumowanie: jedno uruchomienie Full (10^L mnożeń) liczy wszystkie K iloczynów X_QY_Q. Mogą być zupełnie niezależne. Za chwilę wstawimy tę samą macierz do różnych kopii X_Q.

Kafelkowanie: iloczyn $N \times D \times N$ z iloczynów $N_0 \times D \times N_0$

- Tniemy X na **bloki wierszy** po N_0 kolejnych wierszy, a Y na **bloki kolumn** po N_0 kolumn. XY składa się z iloczynów blok wierszy $\times$ blok kolumn, każdy kształtu $N_0 \times D$ razy $D \times N_0$ (krótko: $N_0 \times D \times N_0$).
- $K_0 := \lfloor \sqrt{K} \rfloor$. Grupujemy bloki w **pasy** po K_0 kolejnych bloków. **Kafelek** to pas wierszy wraz z pasem kolumn. Każdy kafelek to **jedno uruchomienie** Full.
- Ustalamy $K_0^2 \leq K$ różnych zbiorów Q rozmiaru m (na rysunku $Q_1, \ldots, Q_9$), po jednym na iloczyn blokowy w siatce, *takie same w każdym kafelku*. X_Q, Y_Q to blok wierszy i blok kolumn iloczynu blokowego z tym Q, a dla pozostałych Q: $X_Q = Y_Q = 0$.
- Lemat 9: jedno Full liczy wszystkie K_0^2 iloczynów kafelka. Wszystkie kafelki dają całe XY.



$N = 6N_0$, $K_0 = 3$: 4 kafelki

kafelek: pas wierszy 2 $\times$ pas kolumn 1,
$K_0^2 = 9$ iloczynów blokowych z jednej
lewej tablicy (pas wierszy) i jednej prawej (pas kolumn)

Dlaczego siatka i ile to kosztuje

- Bloki parujemy w siatkę, a nie dowolnie, więc **lewa tablica kafelka zależy tylko od jego pasa wierszy**, a prawa tylko od pasa kolumn. Pozwoli to współdzielić kodowania między kafelkami (2.4.1).
- Najpierw dopełniamy N do wielokrotności $K_0 N_0$ zerowymi wierszami X i kolumnami Y. To co najwyżej podwaja N, bo $N \geq K_0 N_0$ (pokażemy w 2.4.4).

Koszt. Niech $M := KN_0^2$ będzie liczbą wpisów wyniku we wszystkich K iloczynach kafelka. Ponieważ $K_0 \geq \sqrt{K}/2$:

$$\#\text{kafelków} = \Big(\frac{N}{K_0 N_0}\Big)^2 = \frac{N^2}{K_0^2 N_0^2} \leq \frac{N^2}{(K/4)N_0^2} = \frac{4N^2}{M}.$$

Razem $O\big(\frac{N^2}{M} \cdot L \cdot 10^L\big)$ operacji, czyli $O(L \cdot 10^L/M)$ na wpis XY.

- Przy $L = 10m$ daje to $N^2 \operatorname{poly}(L)$, a przy wybranym przez nas $L = 19m$ około $N^2 D^{0.2}$.
- Mimo to wybieramy $L = 19m$. Dlaczego, zobaczymy w 2.4.3 (rzadkość).
- Kafelkowanie powiększa tylko wymiary zewnętrzne, więc obsługuje mniejsze D niż algorytm Coppersmitha z tej samej tożsamości ($D \approx N^{0.1204}$ zamiast $N^{0.1402}$ [Cop82]). Za to każdy wpis XY to jedno wyjście jednego uruchomienia Full.

Plan

1. Twierdzenie 5: problem i wynik
2. 2.1 Rekurencyjny algorytm Strassena
3. 2.2 Tożsamość Schönhagego
4. 2.3 Mnożenie prostokątne tożsamością Schönhagego (Coppersmith)
5. 2.4 Liczymy tylko chciane wpisy
6. 2.4.4 Dowód Twierdzenia 5
7. Podsumowanie

Cel i dwie modyfikacje

Chcemy policzyć tylko wpisy XY na pozycjach z W, gdzie $|W| \leq N^2/\sqrt{D}$. Algorytm z 2.3 modyfikujemy na dwa sposoby:

1. **Współdzielenie kodowania (2.4.1).** Kodowanie (krok (2)) każdej tablicy wejściowej zależy *tylko od niej*, a każda tablica jest wspólna dla wielu kafelków. Liczymy je **raz** na pas.
2. **Przycinanie drzewa (2.4.2).** W krokach (3)–(4) pomijamy wywołanie rekurencyjne, jeśli żadne z jego wyjść nie prowadzi do wpisu z W.

Potem:

- liczymy odwiedzane liście (2.4.3, **tu jest nowa analiza**),
- sumujemy koszty (2.4.4).

2.4.1 Kodowanie liczymy raz

- Liczby $\Phi_\tau(a)$ i $\Psi_\tau(b)$ mnożone w liściu τ zależą odpowiednio **tylko od** a i **tylko od** b. Liczymy je więc raz na tablicę wejściową, a nie raz na kafelek.
- **Kodowanie** tablicy a to tablica 10^L liczb $\Phi_\tau(a)$ indeksowana liśćmi τ.
- Liczymy ją, uruchamiając Full bez b i bez kroku (4). Z Lematu 7 lewa liczba w liściu τ to $\Phi_\tau(a)$, więc liść ją zapisuje. Koszt to $O(10^L)$, czyli koszt kodowania z 2.3.1.
- Tak samo liczymy kodowanie b, czyli liczby $\Psi_\tau(b)$.
- Liczymy kodowania tablic wejściowych **wszystkich pasów wierszy i kolumn**: $2N/(K_0N_0)$ tablic po 10^L liczb. Każdy kafelek czyta swoje dwa kodowania.

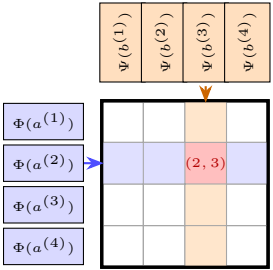
Dlaczego współdzielenie kodowania jest konieczne

- Kodowanie ma

$$10^L = (9+1)^L > \binom{L}{m} 9^{L-m} = M$$

liczb (to jeden wyraz rozwinięcia dwumianu), czyli *więcej niż wpisów kafelka*.

- Kodowanie każdego kafelka osobno kosztowałoby więc więcej niż jedną operację na wpis całego XY, i to zanim w ogóle skorzystamy z W.
- Przy współdzieleniu $2N/(K_0N_0)$ kodowań obsługuje wszystkie $(N/(K_0N_0))^2$ kafelków. Każde kosztuje $O(10^L)$, niezależnie od W.
- Założenie $N \geq D^{18}$ czyni ten koszt pomijalnym (policzymy w 2.4.4).



$a^{(i)}$: lewa tablica pasa wierszy i,
$b^{(j)}$: prawa tablica pasa kolumn j.
Kafelek (i,j) czyta $\Phi(a^{(i)})$ i $\Psi(b^{(j)})$.

2.4.2 Pomijamy niepotrzebne wywołania

- Po policzeniu kodowań pomijamy tę fazę Full. Każdy liść czyta swoje dwie liczby z kodowań, a każde wywołanie wykonuje już tylko dekodowanie z kroku (4).
- Każdemu wywołaniu przekazujemy zbiór S **wyjść, których od niego chcemy**, czyli prowadzących do wpisów z W.
- Wyjścia wierzchołka $\tau_1 \cdots \tau_k$ są indeksowane napisami wyjściowymi długości $L - k$, więc S jest zbiorem takich napisów.

Definicja

Dla zbioru S napisów wyjściowych i zmiennej wyjściowej z **slice** to

$$S_z := \{w' : zw' \in S\}.$$

Tablica na S przypisuje liczbę całkowitą każdemu napisowi z S.

Jeśli U to zbiór przekazany korzeniowi (w końcowym algorytmie są to napisy chcianych wpisów kafelka), to w Lemacie 10 zobaczymy, że S składa się z sufiksów $w_{k+1} \cdots w_L$ tych napisów $w \in U$, do których wierzchołek wnosi wkład.

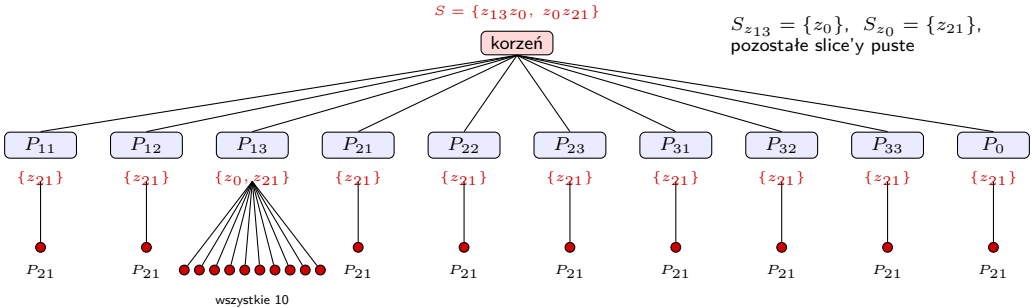
Algorytm Pruned

Pruned$_{\tau_1 \cdots \tau_k}(S)$ dla zbioru S napisów wyjściowych długości $L-k$; zwraca tablicę wierzchołka $\tau_1 \cdots \tau_k$ algorytmu Full ograniczoną do S

1. Jeśli $k = L$: odczytaj $\Phi_\tau(a)$ i $\Psi_\tau(b)$ z policzonych kodowań i zwróć $\Phi_\tau(a)\Psi_\tau(b)$ dla liścia $\tau = \tau_1 \cdots \tau_L$.
2. Dla każdego składnika λ niech S_λ będzie sumą slice'ów S_z po z, do których λ wnosi wkład:
$$S_{P_{ij}} := S_{z_{ij}} \cup S_{z_0}, \qquad S_{P_0} := S_{z_0}.$$
3. Dla każdego λ z $S_\lambda \neq \emptyset$ policz $C_\lambda := \text{Pruned}_{\tau_1 \cdots \tau_k \lambda}(S_\lambda)$, tablicę na S_λ.
4. Zwróć tablicę c na S o slice'ach $c_{z_{ij}} := C_{P_{ij}}$ i $c_{z_0} := \sum_\lambda C_\lambda$, ograniczonych do $S_{z_{ij}}$ i S_{z_0}.

- Ograniczenia w (4) mają sens, bo $S_{z_{ij}} \subseteq S_{P_{ij}}$ i $S_{z_0} \subseteq S_\lambda$ dla każdego λ (z kroku (2)).
- Różnica względem Full: brak kodowania (jest gotowe) i wywołania tylko dla $S_\lambda \neq \emptyset$.

Animacja: Pruned dla $L = 2$, $U = \{z_{13}z_0,\ z_0z_{21}\}$



- Krok (2): $S_{P_{13}} = S_{z_{13}} \cup S_{z_0} = \{z_0, z_{21}\}$, a dla $\lambda \neq P_{13}$: $S_\lambda = S_{z_0} = \{z_{21}\}$. Wywołujemy wszystkie 10 dzieci.
- Pod $\lambda \neq P_{13}$ jest $S = \{z_{21}\}$: niepuste jest tylko $S_{P_{21}} = \{\epsilon\}$ (ϵ to pusty napis). Jeden liść λP_{21}.
- Pod P_{13} jest $S = \{z_0, z_{21}\}$: $S_{z_0} = \{\epsilon\}$, więc wszystkie 10 liści $P_{13}\lambda$.
- Odwiedzonych liści: $10 + 9 = 19$ ze 100. Rzeczywiście: do $z_{13}z_0$ wnosi wkład 10 liści, do z_0z_{21} też 10, a wspólny jest jeden ($P_{13}P_{21}$). Łączny rozmiar zbiorów: $2 + 11 + 19 = 32 \leq (L + 1) \cdot 19$.

Lemat 10: Pruned jest poprawny i odwiedza tylko potrzebne liście

Przypomnienie (2): Mult$(a, b)[w]$ to suma $\Phi_\tau(a)\Psi_\tau(b)$ po liściach wnoszących wkład do w, czyli wybierających P_{ij} na każdym poziomie, gdzie w ma z_{ij}, i dowolny składnik tam, gdzie w ma z_0.

Definicja

Dla zbioru U napisów wyjściowych długości L, Leaves(U) to zbiór liści wnoszących wkład do *jakiegoś* napisu z U.

Lemat 10

Niech a i b będą tablicami wejściowymi, a U zbiorem napisów wyjściowych długości L. Wywołany w korzeniu, Pruned(U) *zwraca* Mult(a, b) *ograniczone do U. Odwiedza dokładnie liście z* Leaves(U)*, a zbiory przekazywane do jego wywołań mają łączny rozmiar co najwyżej* $(L + 1)\,|\text{Leaves}(U)|$.

Dowód Lematu 10

Dowód.

Wartości: indukcja po $L - k$. Każde C_λ to tablica odpowiedniego dziecka w Full ograniczona do S_λ, więc krok (4) to krok (4) algorytmu Full ograniczony do S.

Odwiedzane liście: S_λ składa się z ostatnich $L - k - 1$ zmiennych tych napisów z S, do których *pierwszej* zmiennej λ wnosi wkład. Indukcja po k pokazuje, że zbiór przekazany wierzchołkowi $\tau_1 \cdots \tau_k$ to sufiksy $w_{k+1} \cdots w_L$ napisów $w \in U$, do których ten wierzchołek wnosi wkład, i że wierzchołek jest wywołany *wtedy i tylko wtedy*, gdy ten zbiór jest niepusty. Zatem liść jest odwiedzony $\iff$ wnosi wkład do jakiegoś napisu z U.

Łączny rozmiar: każdemu sufiksowi w zbiorze wierzchołka $\tau_1 \cdots \tau_k$ przypisujemy liść, który przedłuża $\tau_1 \cdots \tau_k$, wybierając P_{ij} tam, gdzie sufiks ma z_{ij}, i P_0 tam, gdzie ma z_0. Wierzchołek wnosi wkład do pewnego $w \in U$ z tym sufiksem, a wtedy ten liść także, więc liść należy do Leaves(U). Liść wyznacza wierzchołek (prefiks długości k) i sufiks (odczytany z jego składników). Zbiory na każdej z $L + 1$ głębokości mają więc łączny rozmiar co najwyżej $|\text{Leaves}(U)|$. □

Kluczowa idea. Każde wywołanie jest wykonane **co najwyżej raz**, niezależnie od tego, ile napisów z U z niego korzysta. Czas zależy od **rozmiaru sumy** zbiorów liści, a nie od sumy ich rozmiarów. Trzeba oszacować tę sumę.

2.4.3 Ile liści wnosi wkład do jednego napisu?

Od teraz U zawiera tylko napisy wyjściowe długości L o zbiorze wewnętrznym rozmiaru dokładnie m (wskazują wpisy $X_Q Y_Q$, Lemat 9). Jest ich

$$M = \binom{L}{m} 9^{L-m} = KN_0^2.$$

- Do jednego napisu o zbiorze wewnętrznym Q wnosi wkład $10^m = D^{\log_4 10} \approx D^{1.66}$ liści: dowolny składnik na każdym z m poziomów Q i P_{ij} na każdym poziomie poza Q.
- To więcej niż D mnożeń potrzebnych do policzenia wpisu wprost jako iloczynu skalarnego!
- Dlatego **nie** wolno liczyć liści każdego napisu osobno. Większość liści napisu wnosi wkład także do wielu innych napisów, a takich liści jest łącznie niewiele.

Definicja

Prywatny liść napisu wyjściowego to ten z jego liści, który wybiera P_0 na każdym poziomie Q.

Nie wnosi on wkładu do żadnego innego napisu. Napis o innym zbiorze wewnętrznym wymaga składnika $\neq P_0$ na jakimś poziomie Q. Napis o tym samym Q, ale innej części zewnętrznej (inne z_{ij} na jakimś poziomie poza Q), wymaga tam innego składnika.

Rząd liścia oraz liczby α_d, β_d

Definicja

Rząd liścia to m minus liczba poziomów, na których wybiera on P_0.

- Liść wnoszący wkład do napisu wybiera P_0 tylko na poziomach zbioru wewnętrznego napisu, więc jego rząd jest ≥ 0. Rząd to liczba poziomów Q, na których liść wybiera któryś z P_{ij}, współdzielonych z iloczynem zewnętrznym.
- Jedynym liściem rzędu 0 wnoszącym wkład do napisu jest jego prywatny liść.

$$\alpha_d := \binom{m}{d} 9^d \qquad \text{liści rzędu } d \text{ wnosi wkład do } \textit{jednego} \text{ napisu}$$

(prywatny liść, w którym na d poziomach Q zamieniamy P_0 na jeden z 9 składników P_{ij}).

$$\beta_d := \binom{L}{m-d} 9^{L-m+d} \qquad \text{liści rzędu } d \text{ jest w } \textit{całym} \text{ drzewie}$$

(wybieramy $m-d$ poziomów z P_0 i jeden z 9 pozostałych składników na każdym innym poziomie). W szczególności $\beta_0 = \binom{L}{m} 9^{L-m} = M$, bo liście rzędu 0 to dokładnie prywatne liście, po jednym na napis.

Podwójne zliczanie

- Liść rzędu d wnosi wkład do $\binom{L-m+d}{d}$ napisów, po jednym na każdy zbiór wewnętrzny rozmiaru m zawierający $m-d$ poziomów, na których liść wybiera P_0. (Dobieramy d z $L-(m-d)$ pozostałych poziomów; część zewnętrzna wynika z P_{ij} liścia.)
- Liczymy pary (napis, liść rzędu d wnoszący do niego wkład) na dwa sposoby:

$$M \cdot \alpha_d = \beta_d \cdot \binom{L-m+d}{d}.$$

Przykład: liście jednego napisu ($L = 6$, $m = 2$)

Napis $w = z_{13}\, z_0\, z_{21}\, z_{33}\, z_0\, z_{12}$, $Q = \{2, 5\}$. Liść to napis składników wybranych na kolejnych poziomach.

P_{13}	P_0	P_{21}	P_{33}	P_0	P_{12}

rząd 0: prywatny liść, $\alpha_0 = 1$, wnosi wkład do 1 napisu

P_{13}	P_{ij}	P_{21}	P_{33}	P_0	P_{12}
P_{13}	P_0	P_{21}	P_{33}	P_{ij}	P_{12}

rząd 1: $\alpha_1 = \binom{2}{1} \cdot 9 = 18$ liści, każdy wnosi wkład do $\binom{5}{1} = 5$ napisów

P_{13}	P_{ij}	P_{21}	P_{33}	P_{ij}	P_{12}

rząd 2: $\alpha_2 = \binom{2}{2} \cdot 81 = 81$ liści, każdy wnosi wkład do $\binom{6}{2} = 15$ napisów

- Liście wyższego rzędu: **więcej na napis** (α_d rośnie), ale każdy **współdzielony przez więcej napisów**.
- Przy $L = 19m$ jest ich **mniej w sumie** (nierówność (5) na następnym slajdzie).

Nierówność (5): liści wysokiego rzędu jest geometrycznie mniej

$$\frac{\beta_d}{\beta_{d-1}} = \frac{\binom{L}{m-d}}{\binom{L}{m-d+1}} \cdot 9 = \frac{9(m-d+1)}{L-m+d}.$$

Dla $L = 19m$ i $1 \le d \le m$: licznik $\le 9m$, mianownik $= 18m + d \ge 18m + 1$, więc

$$\frac{\beta_d}{\beta_{d-1}} = \frac{9(m-d+1)}{L-m+d} \le \frac{9m}{18m+1} < \frac{1}{2}, \qquad \text{zatem} \qquad \beta_d \le 2^{-d} M. \tag{5}$$

- Liczymy **wszystkie** liście rzędu $d > m/9$, niezależnie od tego, czy wnoszą wkład do napisu z U. To tanie, bo β_d maleją geometrycznie.
- To wymaga m **dużo poniżej** $L/10$. Przy $m \approx L/10$ (wybór, przy którym całe XY liczy się tanio, 2.3.4) iloraz byłby bliski 1 dla małych d, więc $\beta_d \approx M$ i nie byłoby oszczędności.
- Małe m sprawia też, że $D = 4^m$ jest małe względem $N_0 = 3^{18m}$ (około $N_0^{0.07}$). Kafelek musi mieścić się w iloczynie, więc N musi być dużą potęgą D. Razem z kosztem kodowań to jest powód założenia $N \ge D^{18}$.

Lemat 11: mało liści wnosi wkład do rzadkiego zbioru wpisów

Od teraz ustalamy $L = 19m$, więc

$$N_0 = 3^{18m}, \qquad K = \binom{19m}{m}, \qquad D = 4^m, \qquad 2^m = \sqrt{D}, \qquad 2^{-m/9} = D^{-1/18}.$$

Lemat 11

Dla każdego zbioru U napisów wyjściowych, których zbiory wewnętrzne mają dokładnie m elementów,

$$|\mathsf{Leaves}(U)| \leq \sum_{d=0}^{m} \min\{|U|\alpha_d,\ \beta_d\},$$

a jeśli $L = 19m$, to

$$|\mathsf{Leaves}(U)| \leq 2^{-m/9}\big(2^m|U| + 2M\big).$$

Dowód Lematu 11: pierwsza nierówność

Dowód (1/2).

Liczbę liści rzędu d w Leaves(U) szacujemy na dwa sposoby:

- przez $|U|\alpha_d$: każdy napis z U „płaci" za swoje liście rzędu d;
- przez β_d: liczbę *wszystkich* liści rzędu d.

Każdy liść z Leaves(U) ma rząd $0 \leq d \leq m$, więc sumując mniejsze z dwóch oszacowań po d, dostajemy pierwszą nierówność. □

Interpretacja (każde z oszacowań osobno):

- $\sum_d |U|\alpha_d = |U| \sum_d \binom{m}{d} 9^d = |U| \cdot 10^m$, czyli koszt liczenia każdego napisu z U osobno;
- $\sum_d \beta_d < 2M$ z (5), czyli koszt policzenia wszystkich M napisów.
- Ani $|U|\alpha_d$, ani β_d nie zależą od tego, *które* napisy są w U.

Dowód Lematu 11: druga nierówność ($L = 19m$)

Dowód (2/2).

Dla rzędów $d \le m/9$ używamy $|U|\alpha_d$, a dla $d > m/9$ używamy β_d.
Rzędy $d > m/9$. Niech d_0 będzie najmniejszą liczbą całkowitą $> m/9$. Z (5):

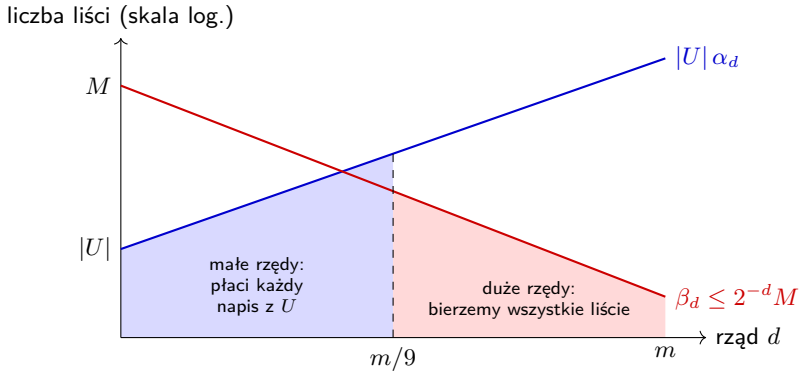
$$\sum_{d>m/9} \beta_d \le M \sum_{d\ge d_0} 2^{-d} = 2\cdot 2^{-d_0} M < 2\cdot 2^{-m/9} M.$$

Rzędy $d \le m/9$. Dają co najwyżej $|U| \sum_{d\le m/9} \binom{m}{d} 9^d$. Ponieważ $9^d = 72^d\, 8^{-d} \le 72^{m/9}\, 8^{-d}$ dla $d \le m/9$,

$$\sum_{d\le m/9} \binom{m}{d} 9^d \le 72^{m/9} \sum_{d=0}^{m} \binom{m}{d} 8^{-d} = 72^{m/9} \Big(\frac{9}{8}\Big)^m = \Big(72\cdot\Big(\frac{9}{8}\Big)^9\Big)^{m/9} < 256^{m/9} = 2^m \cdot 2^{-m/9},$$

bo $72\cdot(9/8)^9 \approx 207{,}83 < 208 < 256$.
Dodając obie części: $|\mathrm{Leaves}(U)| \le 2^{-m/9}\big(2^m|U| + 2M\big)$. □

Rysunek 7: oszacowanie z Lematu 11 (schematycznie)



- Liści rzędu d w Leaves(U) jest co najwyżej $\min\{|U|\alpha_d, \beta_d\}$. $|U|\alpha_d$ rośnie z d aż do $d \approx 0{,}9m$ (do każdego napisu wnosi wkład więcej liści wyższego rzędu), a β_d maleje geometrycznie (liści wyższego rzędu jest mniej).
- Dowód dzieli sumę w $d = m/9$, co nie musi być punktem przecięcia obu oszacowań.
- Całość szacujemy przez zacieniowane pole.

Co mówi Lemat 11 dla gęstości z Twierdzenia 5

Ponieważ $2^m = \sqrt{D}$ i $2^{-m/9} = D^{-1/18}$, druga nierówność to

$$|\text{Leaves}(U)| \leq D^{-1/18}\big(\sqrt{D}\,|U| + 2M\big).$$

Weźmy kafelek, w którym chcemy $|U| = M/\sqrt{D}$ wpisów (gęstość z Twierdzenia 5: $N^2/\sqrt{D}$ z N^2). Wtedy $\sqrt{D}|U| = M$ i

$$|\text{Leaves}(U)| \leq D^{-1/18} \cdot 3M = O\big(M/D^{1/18}\big).$$

- To o czynnik $D^{1/18}$ **mniej niż** M wpisów kafelka.
- I dużo mniej niż $|U| \cdot D = M\sqrt{D}$ mnożeń przy liczeniu każdego chcianego wpisu wprost.
- Oszacowanie jest **liniowe w** $|U|$, więc po zsumowaniu po kafelkach wynik nie zależy od tego, jak chciane wpisy rozkładają się na kafelki.

Plan

1. Twierdzenie 5: problem i wynik
2. 2.1 Rekurencyjny algorytm Strassena
3. 2.2 Tożsamość Schönhagego
4. 2.3 Mnożenie prostokątne tożsamością Schönhagego (Coppersmith)
5. 2.4 Liczymy tylko chciane wpisy
6. 2.4.4 Dowód Twierdzenia 5
7. Podsumowanie

Konsekwencja (a) założenia $N \geq D^{18}$

Ustalamy $D = 4^m$ i $L = 19m$. Mamy $D^{18} = 4^{18m}$, więc potęgi m-te porównujemy, porównując ich podstawy.

(a) Kafelkowanie: $K_0 N_0 \leq N$, więc dopełnienie co najwyżej podwaja N

Z ogólnego oszacowania $\binom{L}{m} \leq (eL/m)^m$: $K = \binom{19m}{m} \leq (19e)^m$. Zatem

$$KN_0 \leq \left(19e \cdot 3^{18}\right)^m \leq \left(4^{18}\right)^m, \qquad \text{bo} \quad 19e \cdot 3^{18} \approx 2{,}001 \cdot 10^{10} < 4^{18} \approx 6{,}872 \cdot 10^{10},$$

a stąd $K_0 N_0 \leq KN_0 \leq D^{18} \leq N$. Dalej N oznacza dopełniony rozmiar.

Konsekwencja (b) założenia $N \geq D^{18}$

(b) Kodowania są tanie

Wykonujemy $2N/(K_0N_0)$ kodowań po $O(10^L)$ operacji. Żeby zmieścić się w budżecie $O(2^{-m/9}N^2)$, potrzeba

$$10^L \leq 2^{-m/9}\, N\, N_0. \tag{6}$$

Zachodzi: $\dfrac{10^L\, 2^{m/9}}{N_0} = \Big(\dfrac{10^{19}\cdot 2^{1/9}}{3^{18}}\Big)^m \leq (4^{18})^m = D^{18} \leq N$, bo $\dfrac{10^{19}\cdot 2^{1/9}}{3^{18}} \approx 2{,}788\cdot 10^{10} < 4^{18}$.

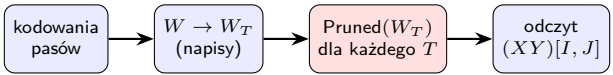
Wtedy łączny koszt kodowań to

$$\frac{2N}{K_0N_0}\cdot O(10^L) \overset{(6)}{\leq} \frac{2N}{K_0N_0}\cdot O\big(2^{-m/9}NN_0\big) = O\Big(\frac{2^{-m/9}N^2}{K_0}\Big) = O\big(2^{-m/9}N^2\big).$$

Algorytm

1. Policz kodowania wszystkich pasów wierszy i pasów kolumn (2.4.1).
2. Każda chciana pozycja $(I, J) \in W$ leży w **jednym** kafelku T i jest wskazana przez **jeden** napis wyjściowy kafelka T. Jego zbiór wewnętrzny to zbiór Q iloczynu blokowego zawierającego (I, J), a wiersz i kolumna to wiersz i kolumna (I, J) wewnątrz tego iloczynu blokowego (2.3.3, 2.3.4).
3. Dla kafelka T niech W_T będzie zbiorem napisów wyjściowych chcianych pozycji w T.
4. Dla każdego kafelka T z $W_T \neq \emptyset$ uruchom Pruned(W_T).
5. Dla każdego $(I, J) \in W$ zwróć wartość przy jego napisie wyjściowym.

Poprawność: z Lematu 9 (napis wskazuje właściwy wpis iloczynu blokowego, czyli $(XY)[I, J]$) i Lematu 10 (Pruned zwraca Mult na W_T).



Koszt

- **Kodowania:** z (6)

$$\frac{2N}{K_0 N_0} \cdot O(10^L) \le \frac{2N}{K_0 N_0} \cdot O\big(2^{-m/9} N N_0\big) = O\big(2^{-m/9} N^2\big).$$

- **Chciane wpisy:** każda pozycja z W trafia do jednego W_T, więc $\sum_T |W_T| \le |W| \le N^2/\sqrt{D} = 2^{-m} N^2$. Kafelków jest co najwyżej $4N^2/M$, więc $\sum_T M \le 4N^2$.
- **Jedno wywołanie** Pruned **ze zbiorem** S**:** $O(L|S|)$ operacji. Każdy krok obsługuje każdy napis z S stałą liczbę razy, a napis ma długość L. Zbiory trzymamy jako posortowane listy, więc slice to spójny fragment, a suma slice'ów to scalanie.
- Z Lematu 10 (łączny rozmiar zbiorów $\le (L+1)|\mathsf{Leaves}|$) wychodzi $O(L^2) = O(\log^2 D)$ operacji **na liść** z $\mathsf{Leaves}(W_T)$.

Z Lematu 11 zsumowanego po kafelkach:

$$O\Big(L^2 \sum_T |\mathsf{Leaves}(W_T)|\Big) \le O\Big(L^2\, 2^{-m/9}\Big(\underbrace{2^m |W|}_{\le N^2} + \underbrace{2 \textstyle\sum_T M}_{\le 8N^2}\Big)\Big) = O\big(L^2\, 2^{-m/9} N^2\big),$$

niezależnie od tego, jak W rozkłada się na kafelki.

Koszt: drobiazgi i wynik końcowy

- **Drobiazgi:** wypisanie K_0^2 zbiorów Q, znalezienie napisów wyjściowych i sortowanie zbiorów W_T kosztują $O(L)$ na zbiór Q, na kafelek i na pozycję z W. Razem to $O(L\,2^{-m/9}N^2)$, bo:
  - $K \le N$,
  - kafelków jest co najwyżej $4N^2/N_0$,
  - $|W| \le 2^{-m}N^2$,
  - $N, N_0 \ge 2^{m/9}$.

Razem

Ponieważ $L = 19\log_4 D$ i $2^{-m/9} = D^{-1/18}$:

$$O\big(2^{-m/9}N^2\big) + O\big(L^2\,2^{-m/9}N^2\big) + O\big(L\,2^{-m/9}N^2\big) = O\left(\frac{N^2\log^2 D}{D^{1/18}}\right). \qquad \blacksquare$$

Rozmiar słowa

Każda liczba w algorytmie jest liczbą całkowitą o module $N^{O(1)}$:

- Wpisy wejścia mają taki rozmiar z założenia.
- Każda liczba w kodowaniu (lub policzona po drodze do niego) to kombinacja ± 1 co najwyżej 7^L wpisów wejścia, bo krok (2) tylko dodaje i odejmuje slice'y.
- Każda wartość policzona przez Pruned to suma co najwyżej 10^L iloczynów dwóch zakodowanych liczb (Lemat 7).
- Z (6): $7^L \leq 10^L \leq 2^{-m/9} N N_0 \leq N^2$.
- Indeksy i pozycje na listach nie przekraczają liczby używanych słów, czyli też $N^{O(1)}$.

Zatem każda liczba ma $O(\log N)$ bitów, czyli mieści się w $O(1)$ słowach modelu word RAM.

Uwaga 12: związek z innymi technikami

Uwaga 12

- Prace o mnożeniu macierzy zwykle liczą tylko *mnożenia*. Tutaj dodawania trzeba kontrolować starannie.
- Kodowania (2.4.1) to algorytm Yatesa dla potęg Kroneckera [Yat37, Goo58], standardowy w rekurencyjnych algorytmach mnożenia macierzy.
- Przycięta rekursja (2.4.2) to analog przycinania FFT [Mar71, SB93], przyciętej inwersji Möbiusa [BHKK10] oraz (po transpozycji) rzadkiej ewaluacji obwodów potęg Kroneckera [AL25], opartej na [Wil24].
- **Nowa** w tym kierunku jest analiza z 2.4.3: przycięta rekursja odwiedza **mało liści**.

Plan

1. Twierdzenie 5: problem i wynik
2. 2.1 Rekurencyjny algorytm Strassena
3. 2.2 Tożsamość Schönhagego
4. 2.3 Mnożenie prostokątne tożsamością Schönhagego (Coppersmith)
5. 2.4 Liczymy tylko chciane wpisy
6. 2.4.4 Dowód Twierdzenia 5
7. Podsumowanie

Kluczowe idee

1. **Rekurencja à la Strassen** z tożsamością Schönhagego (10 mnożeń zamiast 13): L poziomów daje naraz $\binom{L}{m}$ cienkich iloczynów $N_0 \times D \times N_0$.
2. **Błąd E ginie**, bo jego jednomiany mieszają zmienne wewnętrzne z zewnętrznymi (Lemat 9).
3. **Kafelkowanie siatką**: każdy wpis XY to jedno wyjście jednego Full, a tablica wejściowa zależy tylko od pasa.
4. **Kodowanie raz na pas**: kodowanie zależy tylko od X albo tylko od Y, a $N \geq D^{18}$ czyni je pomijalnym.
5. **Przycinanie**: odwiedzamy tylko Leaves(U). Każde wywołanie raz, $O(L^2)$ na liść (Lemat 10).
6. **Liczenie liści według rzędu**: prywatne liście ($d = 0$) plus liście współdzielone. Małe rzędy liczymy na napis, a duże globalnie (β_d maleje geometrycznie przy $L = 19m$). Stąd $O(M/D^{1/18})$ liści na kafelek przy gęstości $1/\sqrt{D}$.